Split method for graph radio-2 vertex colorings

Onat Tevfik Bilgili

February 8, 2026

1 Introduction and Motivation

The L(d1 , ..., dk) labeling problem is a generalization of the vertex coloring problem that has its roots in the channel assignment problem. In the channel assignment problem radio transmitters are to be assigned transmission frequencies such that they do not cause interference among each other. In order to avoid interferences the frequencies of the transmitters must differ by a certain amount depending on the distances between them, whilst using the available frequencies as efficiently as possible.

As wireless communication networks become more widespread, this problem became more significant and it was thought to approach this problem as discrete otimization Problem. This problem of frequency assignment was first modeled by Hale[2] as the T-coloring of graphs.

The transmitters were thought of as the vertices of a graphs and nearby transmitters were connected together by edges. Later Roberts (as mentioned by [4]) proposed a version of this problem where close transmitters must have different frequencies and very close transmitters must have frequencies that differ by a greater amount. This problem was called the L(2,1) labeling problem (also called graph radio-2 coloring problem). It was later proven by Griggs and Yeh [4] that the L(2,1) labeling problem was NP-hard.

As an NP-hard problem integer linear programming formulations have been implemented to solve these problems (first for radio-3 colorings[8] which has later been generalized for radio-k colorings[5]), however these method still suffer from exponential runtime w.r.t. instance sizes and increasingly more difficult to solve depending on graph density[1].

As the exponential runtime scaling for large instances effects a wider range of problems; Vandelli, Ferrari and Dragoni have proposed a Subproblem ParalleL Iterative Technique (shortened as SPLIT) to divide instances of quadratic programming into smaller subproblems and solve them in parallel iteratively[9]. In this paper they present significant results that they have achieved for max-cut and antenna placement problems. Inspired by the results achieved for the antenna placement problem, this paper will summarize the split method and adapt it for radio-2 coloring problem in Section 3. This will later be tested on randomly generated instances in Section 4.

2 Background

2.1 Radio-2 Coloring Problem

We will start this section by defining the radio-2 coloring problem.

Definition 2.1. Let k, d_1 , ..., d_k be natural numbers such that k is non zero. For a positive integer λ a $\lambda - L(d_1, ..., d_k)$ labeling of a graph G is defined as a function f , that maps the vertices of G into the set $\{0, 1, ..., \lambda\}$ for which the following constraint holds true:

$$\forall v, w \in V(G) \text{ s.t. } dist(v, w) = k' \leq k \Rightarrow |f(v) - f(w)| \geq d_{k'}$$

The Problem of finding a minimal λ s.t. there is a $\lambda - L(d_1, ..., d_k)$ labeling is called the $L(d_1, ..., d_k)$ coloring problem. A special case where $\forall i \in \{1, ..., k\} = [k]$ we have $d_i = k - i + 1$ is called the graph radio-k coloring problem. If $k = 2$ we then have the radio-2 or $L(2, 1)$ coloring problem. In this paper we will focus on the radio-2 coloring number.

One of the exact methods to solve the radio-2 coloring problem is to formulate it as an integer linear programming problem. The solutions of these formulations then correspond to an optimal solution of the vertex coloring problem. These integer linear programs can then be solved by an ILP solver, and the solutions can be transformed into colorings. There have been multiple different approaches to this method. In this paper will now consider the assignment based approach suggested by Korze, Shao and Vesel[5] because of it's simplicity and the runtime comparisons made in [1] with other ILP formulations.

These ILP formulations depend on an upper bound on the radio-2 chromatic number (defined analogously to the chromatic number). These can be computed by a polynomial time heuristic that find a feasible radio-2 coloring, such as the one given by Saha and Panigrahi[7]. Let such an upper bound ub be provided. The following is the ILP formulation introduced in [5]:

For each color $i \in \{0, ..., ub\}$ and each vertex $v \in V(G)$ we introduce a binary variable $x_{v,i} \in \{0, 1\}$. This variable $x_{v,i}$ will attain value 1 iff. the vertex v has the color i in the radio-2 coloring. Furthermore we introduce a variable λ which will represent the radio-2 chromatic number. We will try to minimize this variable. This leads to the following ILP formulation:

$$\text{minimize } \lambda$$

subject to:

$$\begin{gathered}
\textstyle\sum_{i=0}^{ub} x_{v,i} = 1, \forall v \in V(G) \\
x_{v,i} + x_{u,i} \leq 1, \forall i \in \{0, ..., ub\}, dist(u, v) = 2 \\
x_{v,i} + x_{u,j} \leq 1, \forall i, j \in \{0, ..., ub\} \text{ s.t. } |i - j| \leq 1 \text{ and } \{u, v\} \in E(G) \\
i \cdot x_{v,i} \leq \lambda, \forall i \in \{0, ..., ub\}, \forall v \in V(G) \\
x_{v,i} \in \{0, 1\}, \forall i \in \{0, ..., ub\}, \forall v \in V(G)
\end{gathered}$$

The first constraint ensures that every vertex has exactly one color. The second constraint ensures that vertices u and v at distance 2 have distinct colors. The third constraint ensures that adjacent vertices have colors that differ by at least 2. And the fourth constraint ensures that the radio-2 chromatic number is greater than or equal to every color used in the coloring. As such an optimal result of this ILP corresponds to an optimal radio 2 coloring of a graph G.

2.2 SPLIT Method

Now that the radio-2 coloring problem has been defined and ILP formulations are introduced, we will now move on to the SPLIT framework for quadratic programming by Vandelli, Ferrari and Dragoni[9]. This method works independent of the solver can be used for both continious and integer/binary variables. It is primarily designed for unconstrained problems, however certain kinds of linear constraints can be built into this framework as well.

Quadratic programming is the problem of minimizing or maximizing a quadratic function over a set of variables. W.l.o.g consider the minimization of the cost function in the form:

$$H(X) = X^T Q X + q^T x$$

where the diagonal entries of the symmetric square matrix Q are zero. We can interpret Q as the adjecency matrix of an undirected weighted graph on n vertices. Then non zero entries of Q correspond to the edges of a graph G. Using this idea H can be rewritten as:

$$H(X) = \textstyle\sum_{\{i,j\} \in E(G)} q_{i,j} \cdot x_i \cdot x_j + q^T X$$

We begin partitioning our problem into smaller subproblems by partitioning $V(G)$ into K clusters. $G_k = (V_k, E_k)$ denotes the subgraph induced by vertices in the cluster k, and $E_{k,l}$ denotes the edges between subgraphs G_k and G_l. Furthermore let X_k denote variables corresponding to vertices V_k. Observe that the target function takes the following form on each cluster of vertices and corresponding variables:

$$H_k(X_k, D_k) = \sum_{\{i,j\}\in E_k} q_{i,j} \cdot x_i \cdot x_j + \sum_{i\in V_k} (q_i i + d_i^{(k)}) \cdot x_i \text{ where}$$
$$d_i^{(k)} = \sum_{l\in[K]-k} \sum_{(i,\alpha)\in E_{k,l}} q_{i,\alpha} \cdot x_\alpha$$

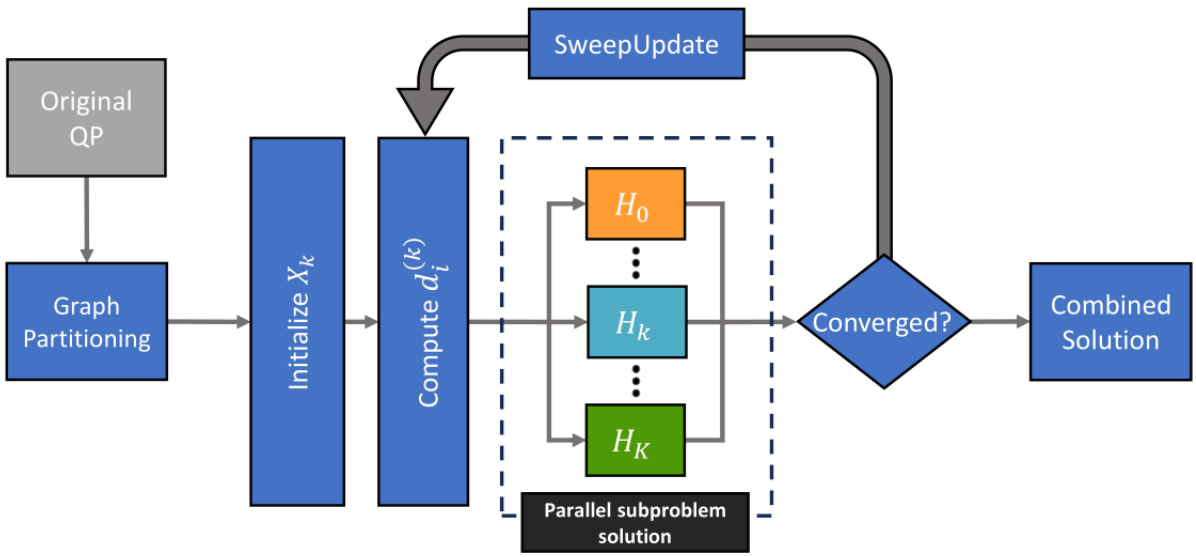
Using this decomposition of G and H the minimization problem can be rewritten as follows:

$$min_{X,D} \sum_k^K (H_k(X_k, D_k) - 0.5 \cdot (\sum_{i\in V_k} d_i^{(k)} x_i)$$

subject to:

$$d_i^{(k)} = \sum_{l\in[K]-k} \sum_{(i,\alpha)\in E_{k,l}} q_{i,\alpha} \cdot x_\alpha, \forall i \in V_k, k \in [K]$$

This method of formulating the cost function locally in each cluster gives the basis for the iterative SPLIT algorithm fo unconstrained quadratic minimization problems. It starts with an initial solution $X^{(0)}$, calculates the local variables d_1 based on the current values of X, runs the local minimization problem on each cluster in parallel and repeats until some convergence criteria is fulfilled. The following image[9] visualizes this algorithm.



A sweep update routine which scrambles the variables randomly can be implemented. This process keeps the random change of variables if it leads to a better value under the cost function. Since this update process is independent of the clustering, it is helps at mitigating the risk of converging to local minima of the iteration process.

The paper "Parallel splitting method for large-scale quadratic programs" tests this algorithm by using CPLEX exact and approximative Quadratic program solver[3] on max-cut and antenna placement problems. There they were able to consistently achieve an approximation ratio better than 99% of the optimal solution whilst on avarage being faster than cplex approximate solution.

The antenna placement problem is the problem of placing n antennas on different possible locations of an area, such that the intersection between the areas covered by those antennas is minimal. The resemblence between the origin of radio coloring and antenna placement problem has motivated the use of similiar formulations for radio colorings, in order to achieve similar results.

3 Adaptation of split for radio-2 colorings

In the previous section of this paper we had defined the radio-2 coloring problem. Here we will define a quadratic penalty relaxation formulation of radio-2 coloring. Here we use a quadratic penalty function to minimize the number of pairs of vertices that violate the radio-2 conditions. We use variables$x_{v,i}$ and λ similar to the ilp formulation from chapter 2.1. Let $\epsilon > 0$, the quadratic penalty formulation is as follows:

$$\text{minimize}$$
$$f(x) = \lambda + \sum_{\{u,v\}\in E(G), u<v} \sum_{i=0}^{ub} \sum_{j=max(i-1,0)}^{min(i+1,ub)} \epsilon \cdot x_{v,i} \cdot x_{u,j} + \sum_{u,v\in V(G), dist(u,v)=2, u<v} \sum_{i=0}^{ub} \epsilon \cdot x_{u,i} \cdot x_{v,i}$$

subject to

$$\sum_{i=0}^{ub} x_{v,i} = 1, \forall v \in V(G)$$
$$i \cdot x_{v,i} \leq \lambda, \forall i \in \{0, ..., ub\}, \forall v \in V(G)$$
$$x_{v,i} \in \{0, 1\}, \forall i \in \{0, ..., ub\}, \forall v \in V(G)$$

The target function punishes every pair of vertices whose colors violate radio-2 conditions by a value of ϵ. Whilst applying SPLIT for radio-2 colorings we are going to partition the graphs vertices first and then decide the partitioning of optimization variables based on the partitioning of the vertices. This way all the constraints of the problem only have variables form exactly one cluster. Thus these constraints do not need to be relaxed for the split algorithm.

The same can be told for certain terms of local the target function f_k as well. If a term $\epsilon \cdot x_{v,i} \cdot x_{v,j}$ of f only has variables from cluster k this term can be restricted to the constraint $x_{v,i} + x_{u,j} < 1$ for the local minimization problem on the cluster k. So only remaining terms $\epsilon \cdot x_{v,i} \cdot x_{v,j}$ have exactly one variable from the cluster variables, and the other factor is read from the previous non-cluster variables. Thus the local minimization problem becomes an ILP, and can be solved by an ILP solver. So that more than one clusters deal with the problem of minimizing the radio-2 chromatic number, we introduce the variable λ_k in each cluster. We add a linear term λ_k in each local target function f_k and remove the previous global variable λ

We compute the clustering by assigning weights to the edges of power graph G^2. We assign weight 2 to the original edges of G and weight 1 to the rest of the edges. We can then run a spectral clustering algorithm the same way it is done for the associated graphs of max-cut and antenna placement problems[9] in the paper that introduces SPLIT.

After a coloring has been obtained at the end of each split iteration, a sweep update process can be implemented as follows: A random subset of vertices are picked and have their colors changed randomly. If this leads to a lower (largest color used) + $\epsilon\cdot$ # {pairs violating radio-2 conditions} we keep the changes. Otherwise reject it.

The best result that has been observed throughout the iterations is returned. This way we have an implementation of SPLIT for radio-2 coloring problems making use of integer linear programming.

4 Implementation and results

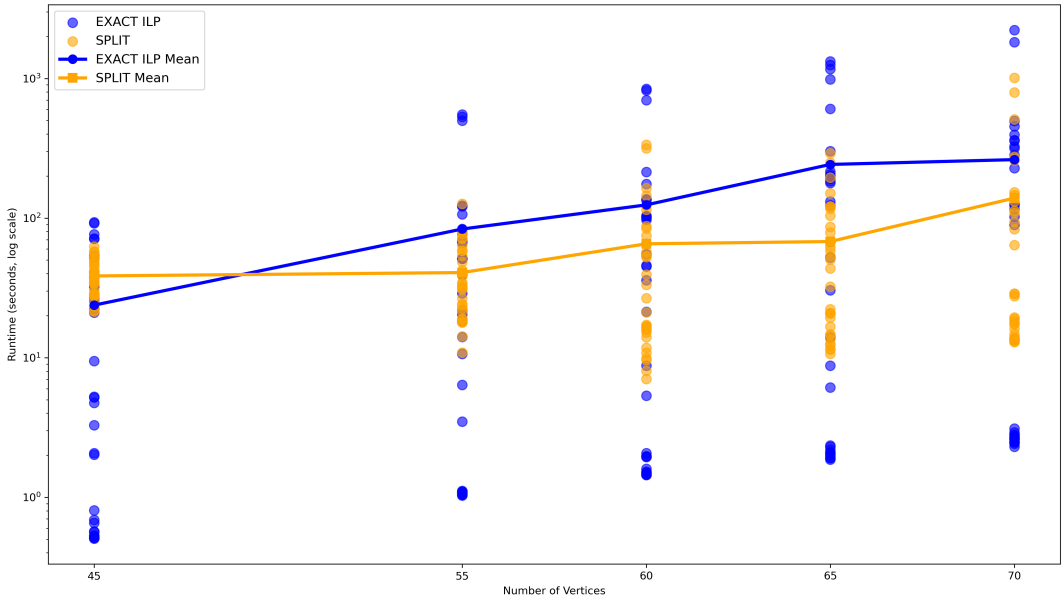
In this section we will test SPLIT for radio-2 coloring on randomly generated instances. The algorithms are implemented in C++, using the graph libraries from the book Algorithmic Mathematics by Vygen & Hougardy [10]. The ILPs were solved by version 22.1 of CPLEX[3] and for the clustering problem python code has been embedded into C++ using the scikit-learn[6] package.

The test instances are randomly generated undirected graphs, having 45, 55, 60, 65 or 70 vertices. For each number of vertices a total of 30 instances were generated, where half of the generated graphs have an edge density of 35% and the rest of them 75%.

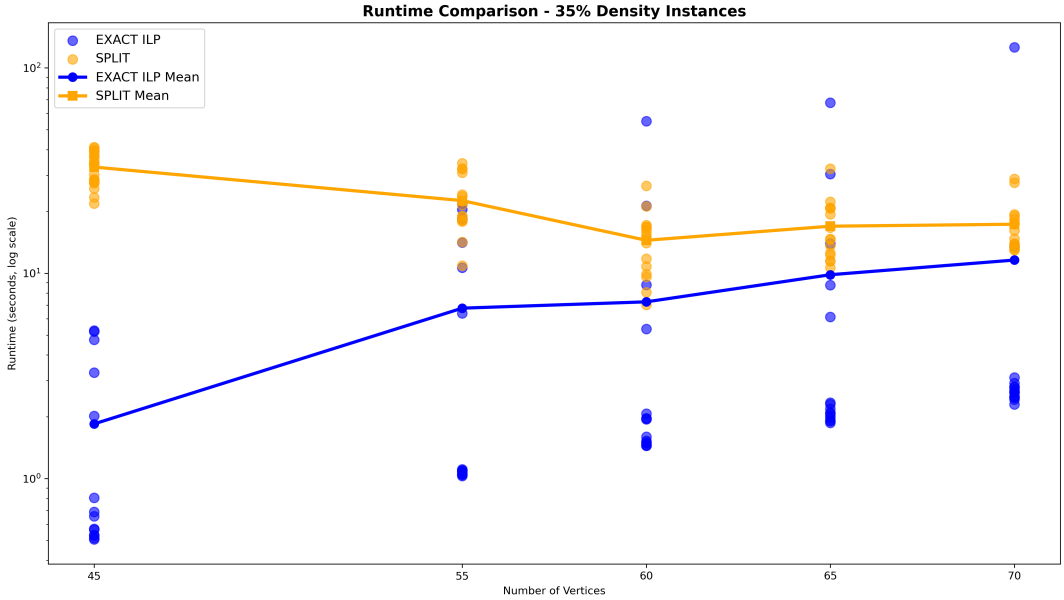
The SPLIT algorithm was run on 4 cluster, made to terminate after 20 iterations or after the evaluation of the iterative results had not changed more than 3% in 5 consecutive iterations. To ensure feasibility, a penalty of ub was added for each condition-violating pair of vertices.

All the instances were tested on a system equipped with an AMD Ryzen 5 5600X processor and 32 GB of ddr4 memory running on version 6.17.0-14-generic of the linux Kernel.

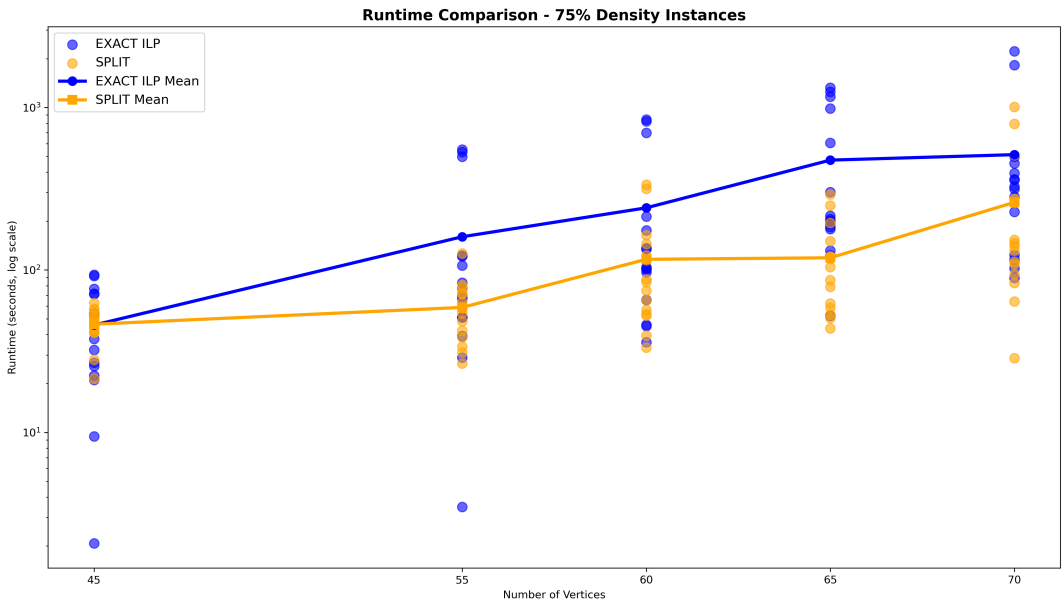
The following graph shows the runtime of SPLIT and exact ILP based implementations across all of the instances. Whilst ILP based solver was faster than SPLIT based solver on instances with 45 vertices, the SPLIT based solver achieved better average runtime compared to ILP based solver as the number the number of vertices increased. A significant factor effecting the runtime were the density, so for further analysis we will distinguish between densities. Instances with lower density were significantly faster to solve by both of these methods.

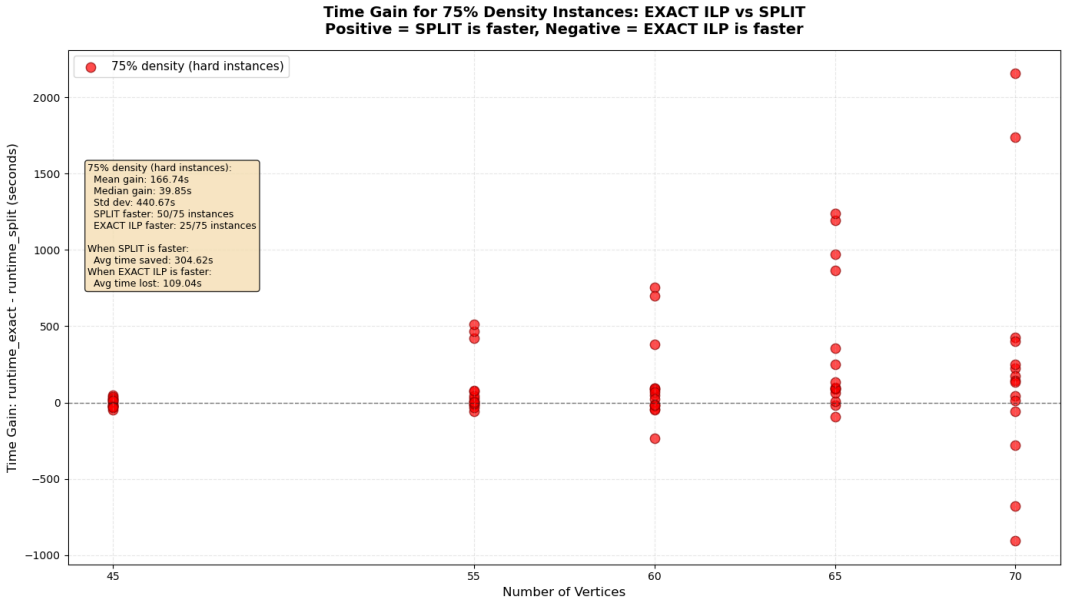


We will first start with 35% density instances. We can observe that across instances of low density we consistently get worse runtime as SPLIT requires multiple iterations to decide at has converged to a desirable solution, however this difference decreased as the number of vertices got larger.

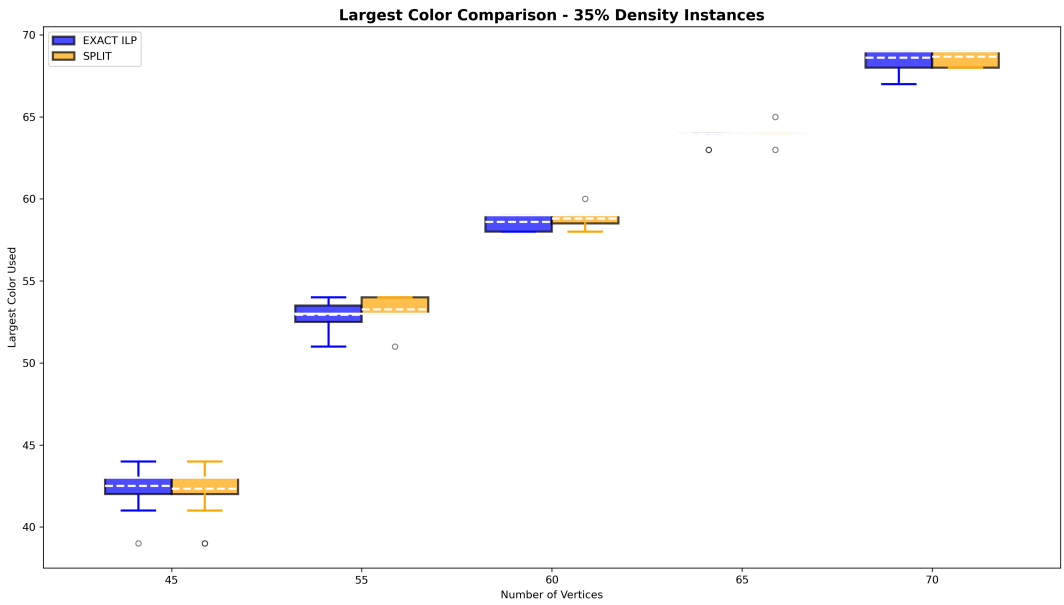


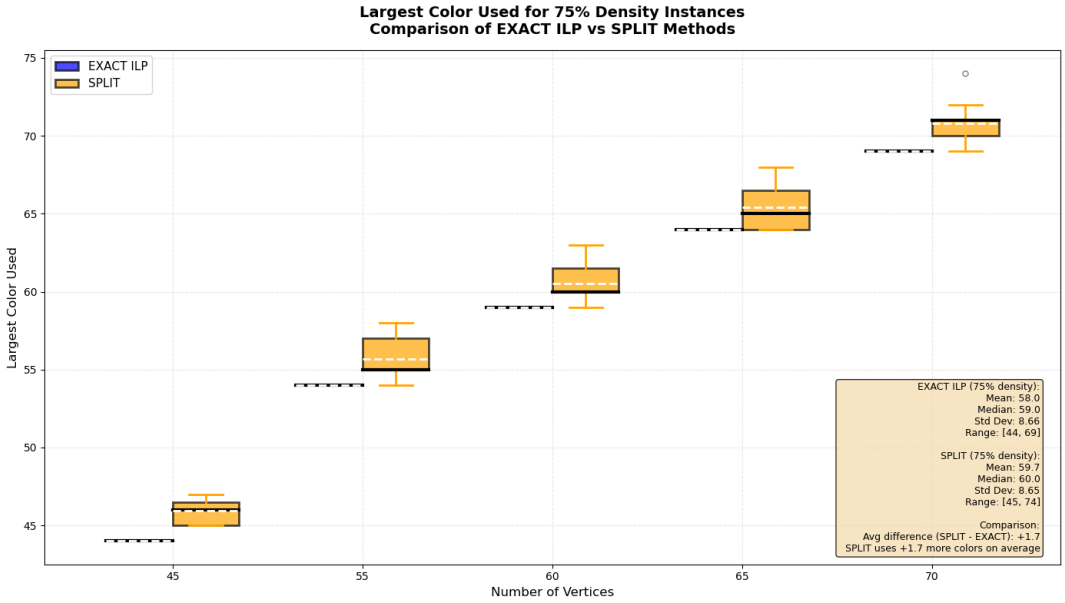
For instances of 75% density we can see that SPLIT was consistently faster. In 50 out of 75 instances SPLIT was faster, with the time gain becoming more significant as the number of vertices increased. On one particular example SPLIT lead to a runtime gain of over 2000 seconds.

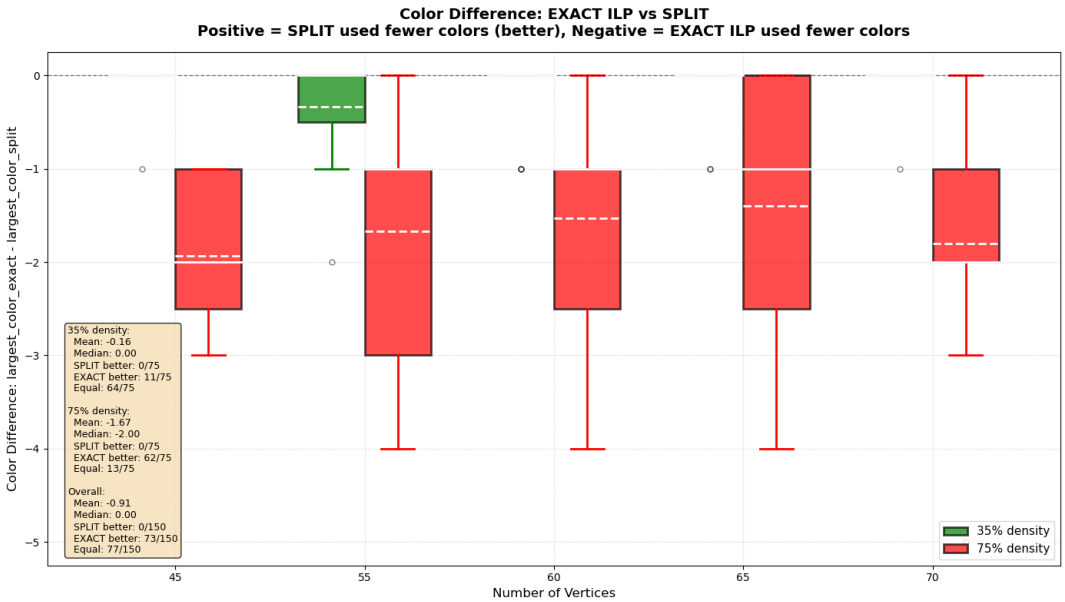




Now that runtime gains are covered, the quality of results produced by SPLIT will be represented. The following graphs visualize the largest color used in the colorings produced by SPLIT and ILP based methods. Here we can observe that the results produced by SPLIT on lower density instances is closer to those produced by ILP based solvers. Since our implementation of SPLIT only returns feasible solutions after end of processes, allowing infeasible solutions depending on the use case may lead to better results. Despite all of problems of SPLIT across dense instances, it consistently attains an approximation ratio over 90%.


Largest Color Comparison - 35% Density Instances
EXACT ILP
SPLIT
Largest Color Used
Number of Vertices


Largest Color Used for 75% Density Instances
Comparison of EXACT ILP vs SPLIT Methods
EXACT ILP
SPLIT
EXACT ILP (75% density):
Mean: 58.0
Median: 59.0
Std Dev: 8.66
Range: [44, 69]
SPLIT (75% density):
Mean: 59.7
Median: 60.0
Std Dev: 8.65
Range: [45, 74]
Comparison:
Avg difference (SPLIT - EXACT): +1.7
SPLIT uses +1.7 more colors on average
Largest Color Used
Number of Vertices


Color Difference: EXACT ILP vs SPLIT
Positive = SPLIT used fewer colors (better), Negative = EXACT ILP used fewer colors
35% density:
Mean: -0.16
Median: 0.00
SPLIT better: 0/75
EXACT better: 11/75
Equal: 64/75
75% density:
Mean: -1.67
Median: -2.00
SPLIT better: 0/75
EXACT better: 62/75
Equal: 13/75
Overall:
Mean: -0.91
Median: 0.00
SPLIT better: 0/150
EXACT better: 73/150
Equal: 77/150
35% density
75% density
Color Difference: largest_color_exact - largest_color_split
Number of Vertices

5 Conclusion

In this paper we have adapted SPLIT method for large scale quadratic programming to the graph radio-2 coloring problem by using ILP formulations on clusters of vertices and adding quadratic penalty cost functions. The results show a significant time gain across dense instances whilst maintaining a high approximation ratio. Although the result among sparse instances, these were still significantly easier to show and results suggest runtime loss across sparse instances get better with increasing number of vertices. Future work may focus on improving solution quality through adaptive penalty strategies, refined clustering methods, or allowing controlled infeasibility during intermediate iterations to further enhance performance.

References

[1] Bilgili, O. T., The L(2,1)-Labeling Problem with and without color lists. Bachelor's Thesis, Rheinische Friedrich-Wilhelms-Universität Bonn, (2024)

[2] Hale, W. K., "Frequency assignment: Theory and applications," in Proceedings of the IEEE, vol. 68, no. 12, (1980) 1497-1514

[3] IBM ILOG Cplex. User's Manual for CPLEX V22.1. 2022. URL:https://www.ibm.com/docs/en/icos/22.1.1?topic=mc-what-is-cplex.

[4] Griggs, J.R. and Yeh, R.K, Labeling Graphs with a condition at distance 2, SIAM Journal on Discrete Mathematics, volume 5, number 4, (1992) 586-595

[5] Korze, D., Shao, Z. and Vesel A., New results on radio k-labelings of distance graphs, Discrete Applied Mathematics, volume 319, (2022) 472-479

[6] Pedregosa, F. et al., Scikit-learn: Machine learning in Python. Journal of machine learning research, 12(Oct), (2011) pp.2825–2830.

[7] Saha, L. and Panigrahi P., A new graph radio k-coloring algorithm, Discrete Mathematics, Algorithms and Applications, volume 11, number 1, (2019)

[8] Shao, Z. and Vesel, A., Integer linear programming model and satisfiability test reduction for distance constrained labelings of graphs: the case of L(3,2,1)labelling for products of paths and cycles. IET Communications, 7: (2013) 715-720

[9] Vandelli, M., Ferrari, F., Dragoni, D. Parallel splitting method for large-scale quadratic programs. Quantum Computing Solutions, Leonardo S.p.A., (2025)

[10] Hougardy, S. and Vygen, J.,Algorithmic Mathematics, (2016) 80-83